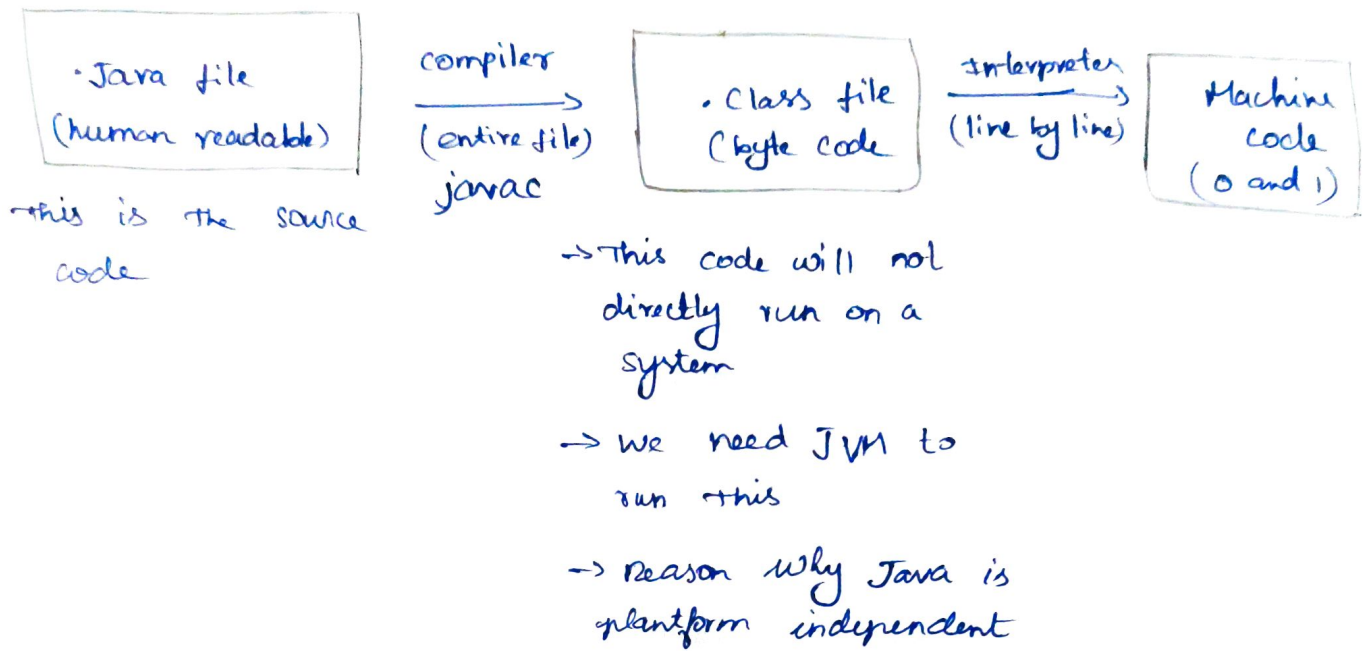


Introduction to JAVA :-

How JAVA code executes



More about platform independent :

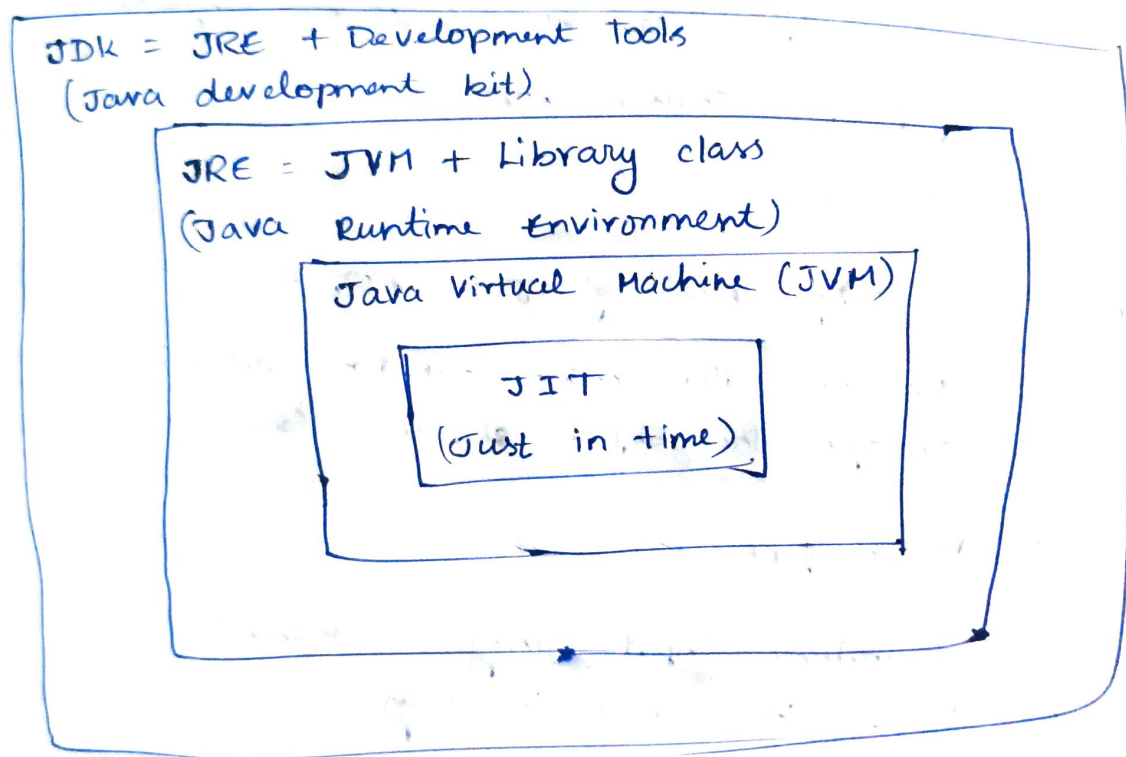
- It means that byte code can run on all operating systems.
- We need to convert source code to machine code so computer can understand.
- compiler helps in doing this by turning it into executable code.
- the executable code is a set of instructions for the computer.
- after compiling C/C++ code we get .exe file which is platform dependent.

→ In java we get bytecode, JVM converts this to machine code.

→ Java is platform independent but JVM is platform dependent.

Architecture of JAVA :-

JDK Vs JRE Vs JVM Vs JIT



Important Notes:-

- * Java uses both a compiler (javac) and an interpreter / JIT compiler (inside the JVM).
- * JDK = JRE + development tools (such as javac)
- * JRE = JVM + java libraries needed to run programs.
- * JVM = class loader + bytecode verifier + execution engine + memory management components.
- * JIT compiler improve performance by frequently executing bytecode into native machine code.

Java program

File name = public class Name

```
public class Main {  
    public static void main (String[] args) {  
        System.out.println("Hello world");  
    }  
}
```

public → access modifier (can be accessed from anywhere)

class → group of methods & properties

main → class name

main function → entry point of java program

Static → to run the main function without creating the object of the class

void → Return type

String[] ^{variable name} arg → whatever input giving in terminal, it will be stored in array. (command line arguments).
↓
array of String (datatype)

primitive datatypes

* store simple value

* Built into java

* faster

* They are not object

e.g:

1) int

2) float

3) long

4) double

5) boolean

6) char

7) short

8) byte

Type casting & Type conversion:

Type casting $\rightarrow$ manual conversion

Type conversion $\rightarrow$ automatic conversion

Java automatically converts a smaller datatype into bigger datatype (Type conversion).

byte $\rightarrow$ short $\xrightarrow{\text{char}}$ int $\rightarrow$ long $\rightarrow$ float $\rightarrow$ double
(smaller $\rightarrow$ bigger)

Explicit conversion:

```
double x = 9.8;
```

```
int y = (int)(x);
```

output $\Rightarrow$ 9

Type promotion:

```
byte a = 40;
```

```
byte b = 50;
```

```
byte c = 100;
```

```
int d = a * b / c;
```

output $\Rightarrow$ 20

a * b gives 2000 which can't store in byte, so (byte * byte $\Rightarrow$ byte) Java automatically converts into integer in expression.

Java automatically converts smaller datatypes into larger datatypes while performing expressions or operations.

Functions/ methods:

functions inside a class $\Rightarrow$ methods
in java every functions are inside classes.

* reusable block of code

* create a func call it
whenever needed.

Syntax:

```
return type function_name(para) {  
    // function body  
}
```

Important points:

- 1) java is always pass by value
- 2) primitives ^{variable} store actual values.
 $\rightarrow$ method call copies the value.
- 3) Reference variables store references
e.g. `Student S = new Student();`
S store the reference (address) of the object
 $\rightarrow$ method call copies the reference value.
- 4) Two reference variables can point to the same object.

a - - - - - $\rightarrow$ object

b - - - - -

* modifying the object through either reference
affects the same object.

Array:-

Syntax:-

$\text{datatype} [\text{size}] \text{ variable_name} = \text{new } \text{datatype} [\text{size}];$

→ datatype represents which type of data stored.

→ all the type of data in the array should be same.

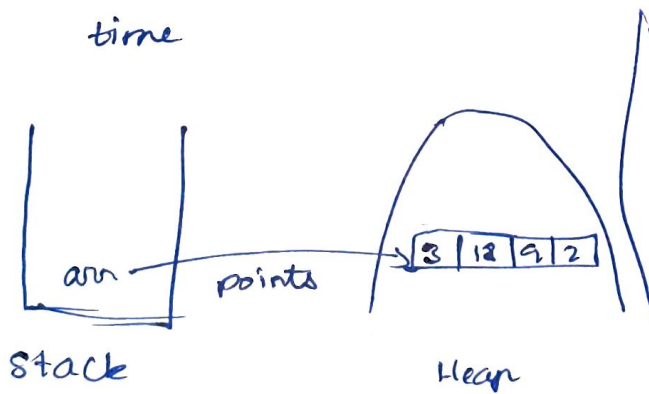
Internal working of an array:

$\text{int} [\text{size}] \text{ arr} = \text{new } \text{int} [\text{size}]$

↑ datatype ↑ ref. variable

happens at compile time

new is used to create an object
↓ (Dynamic memory allocation)
assigning memory at run time
→ creating the object in heap memory



The array itself:-

- * is a continuous block of memory
- * But it stores references (i.e. memory addresses) to objects

The actual objects:-

- * Can be stored anywhere in the heap

* Array objects are in heap (all objects).

* heap objects are not continuous. (depends on JVM)

* Dynamic memory allocation

Hence, In java array ^{objects} elements may not be continuous.

3	18	9	10	17
0	1	2	3	4

`print(arr[0]) = 3`

`arr[2] = 9` → [3 | 18 | 7 | 10 | 17]